CYBERSECURITY

VORTEXTECH INTERNSHIP REPORT-WEEK 1

Prepared by: Noor Fatima

Task 1: 8th September, 2026

OWASP Top 10: 5 Critical Web Application Security Risks

VortexTech Cyber Security Internship - Week 1

Prepared by: Noor Fatima

Track: Cyber Security

Internship Date: September 2026

INTRODUCTION:

The OWASP Top 10 is one of the most widely recognized awareness documents in web application security. It identifies the most critical risks that developers and security professionals should understand when building and protecting web applications. This report researches five vulnerabilities from the current OWASP Top 10:2025 list.

The five selected categories are:

1. A01:2025: Broken Access Control

2. A02:2025: Security Misconfiguration

3. A03:2025: Software Supply Chain Failures

4. A05:2025: Injection

5. A07:2025: Authentication Failures

1. A01:2025 - Broken Access Control

What is it?

Broken Access Control happens when an application does not properly enforce what different users are allowed to do. Every user should only be able to access the information and actions that match their role and permissions.

For example, a normal customer should not be able to access an administrator dashboard. Similarly, one user should not be able to view or edit another user's private account simply by changing an ID number in a URL.

In simple terms, broken access control means the application fails to answer the question: "Is this user actually allowed to do this?

Which results in data breaches, privilege escalation, or account takeover.

How could an attacker exploit it?

An attacker can exploit it by using multiple techniques such as:

- Change an ID in a URL, such as `/profile/101` to `/profile/102`, to access another user's information.
- Directly visit hidden administrator pages.
- Modify API requests to perform actions that should be restricted.
- Change parameters or tokens in an attempt to gain higher privileges. Send requests to endpoints that do not properly check authorization.

If the server trusts the request without verifying permissions, the attacker may gain access to sensitive data or perform unauthorized actions.

Real-world example:

A well-known example of access-control-related security failures occurred in the Facebook.

Basically Facebook had a feature called “View As”, which allowed users to see how their Facebook profile looked to other people.

In 2018, attackers discovered a security weakness in this feature. Because of this weakness, they were able to steal special login keys called access tokens.

An access token like a temporary digital key that proves you are logged into your Facebook account. If a hacker steals this key, they may be able to access your account without knowing your password.

Around 50 million Facebook accounts were directly affected using this flaw.

How can developers prevent it?

Developers can prevent it if they:

- Enforce authorization checks on the server side.
- Follow the principle of least privilege.
- Deny access by default unless permission is explicitly granted.
- Verify that users own the records they are trying to access.
- Use role-based or attribute-based access controls where appropriate.

- Protect all API methods, including GET, POST, PUT, and DELETE.
- Log repeated authorization failures and monitor suspicious activity.

2. A02:2025 - Security Misconfiguration

What is it?

Security Misconfiguration occurs when an application, server, database, cloud service, or other technology is not configured securely. A system may contain strong security features, but if those features are set up incorrectly, disabled, or left at insecure default settings, attackers may still find a way in.

For Example:

• Default usernames and passwords still being active.

• Unnecessary services or ports being exposed.

• Detailed error messages revealing sensitive information.

• Cloud storage accidentally made public.

• Missing security headers.

• Excessive permissions granted to users or services

In simple terms, security misconfiguration means the technology may be secure, but the way it was set up is not.

How could an attacker exploit it?

An attacker can exploit it by scanning a target's:

- Open services that should not be publicly available.
- Default login credentials.
- Public cloud storage buckets.
- Exposed administrative panels
- Detailed error messages containing software versions or internal paths
- Incorrectly configured permissions

The attacker can then use this information or exposure as an entry point into the system.

Real-world example:

The 2019 Capital One data breach is commonly discussed as an example involving cloud security configuration and infrastructure weaknesses. An attacker exploited a vulnerability involving a web application firewall and obtained credentials that allowed access to data stored in AWS infrastructure.

The breach affected more than 100 million individuals in the United States and Canada and exposed sensitive personal information.

This case showed how cloud environments require careful configuration and strict permission management.

How can developers prevent it?

To prevent this:

- Remove unnecessary services, accounts, pages, and features.
- Change all default credentials.
- Apply secure configuration baselines.
- Use automated configuration and security checks.
- Restrict cloud permissions according to least privilege.
- Avoid exposing detailed error messages to users.
- Regularly review security headers and server settings.
- Separate development, testing, and production environments

3. A03:2025 - Software Supply Chain Failures

What is it?

Modern applications depend on many external components, including open-source libraries, third-party packages, build tools, cloud services, and software vendors.

A Software Supply Chain Failure occurs when attackers compromise one of these trusted components or the process used to build and distribute software.

This is dangerous because an organization may have strong internal security but still become compromised through software or services it trusts.

How could an attacker exploit it?

An attacker may:

- Compromise a software vendor's update system.
- Insert malicious code into a trusted software package.
- Take over a dependency used by many applications.
- Compromise build or deployment infrastructure.
- Trick developers into installing a malicious package with a similar name.

Once the malicious component reaches victims, the attacker may gain access to many organizations at the same time.

Real-world example:

The SolarWinds supply-chain attack, discovered in 2020, is one of the most significant examples of this type of attack.

Attackers compromised the SolarWinds software build process and inserted malicious code into legitimate Orion software updates. These trusted updates were then distributed to thousands of customers, including government agencies and major companies.

The incident demonstrated that trusting a legitimate vendor or software update does not guarantee that the software is safe if the supply chain itself has been compromised

How can developers prevent it?

Organizations should:

- Maintain an inventory of software dependencies.
- Verify the integrity and origin of packages and updates.
- Keep dependencies updated and remove unnecessary ones.
- Secure build and deployment systems.
- Use code signing and integrity verification where possible.
- Review third-party vendor security practices.
- Monitor for suspicious behavior from trusted software.
- Limit the permissions granted to third-party components.

4. A05:2025 - Injection

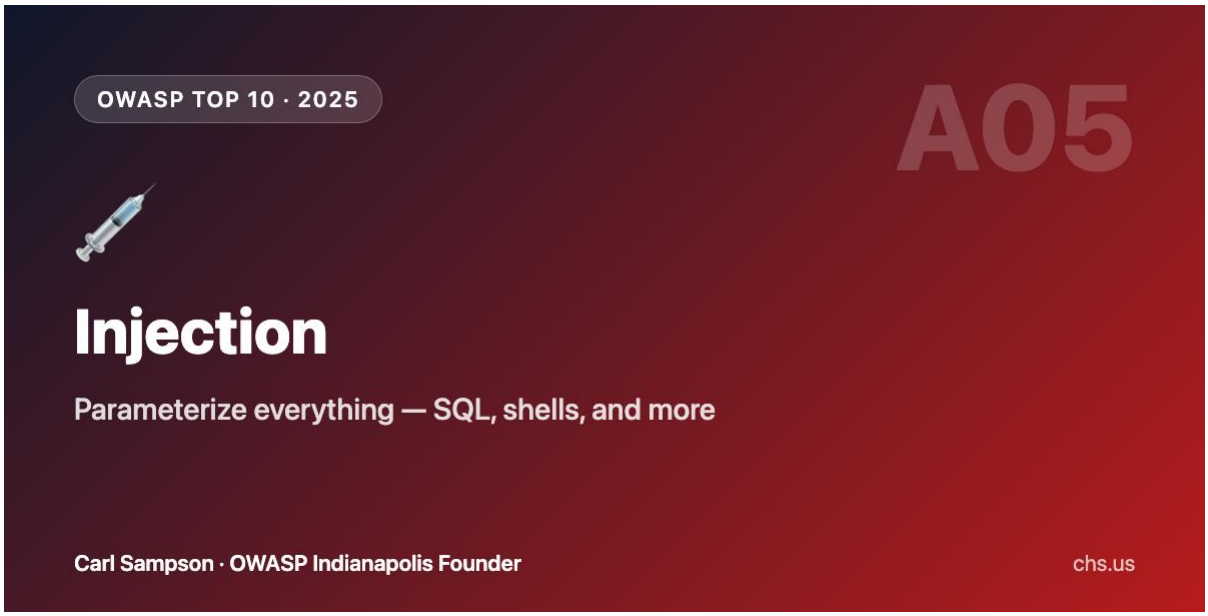
What is it?

Injection occurs when an application accepts untrusted input and sends it to another system without safely handling it.

- Common forms include:
- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)
- Command Injection
- LDAP Injection

For example, a website may take information entered by a user and directly place it inside a database query. If the input is not handled safely, an attacker may insert special commands instead of normal data.

In simple terms, injection happens when an application cannot distinguish between data provided by a user and instructions that should be executed by a system.



How could an attacker exploit it?

For SQL Injection, an attacker might enter specially crafted input into a login form or search field.

If the application builds database queries by directly combining user input with SQL commands, the attacker's input may change the meaning of the query.

Depending on the vulnerability, an attacker could:

- Bypass authentication.
- Read sensitive database information.
- Modify or delete records.
- Access administrator data
- In severe cases, take further control of connected systems.

Real-world example:

In 2015, telecommunications company TalkTalk suffered a major data breach after attackers exploited a SQL Injection vulnerability in its website.

The attack resulted in the exposure of customer information and caused significant financial and reputational damage to the company.

The incident remains a widely cited example of how a relatively well-known web vulnerability can still have serious consequences when secure coding practices are not followed.

How can developers prevent it?

Developers should:

- Use parameterized queries or prepared statements.
- Never directly concatenate user input into SQL queries.
- Validate input based on expected formats.
- Use safe APIs and ORM frameworks correctly.
- Apply the principle of least privilege to database accounts.
- Escape output appropriately to prevent XSS.
- Conduct regular security testing and code reviews.

5. A07:2025 - Authentication Failures

What is it?

Authentication is the process of verifying that a user is really who they claim to be. Authentication Failures occur when login and identity verification systems are weak or incorrectly implemented. This can allow attackers to take over accounts or impersonate legitimate users.

Examples include:

- Weak passwords being accepted.
- Password reuse.
- Missing multi-factor authentication (MFA).
- Poor session management.
- Credentials being exposed or leaked.
- Login systems lacking protection against automated attacks.

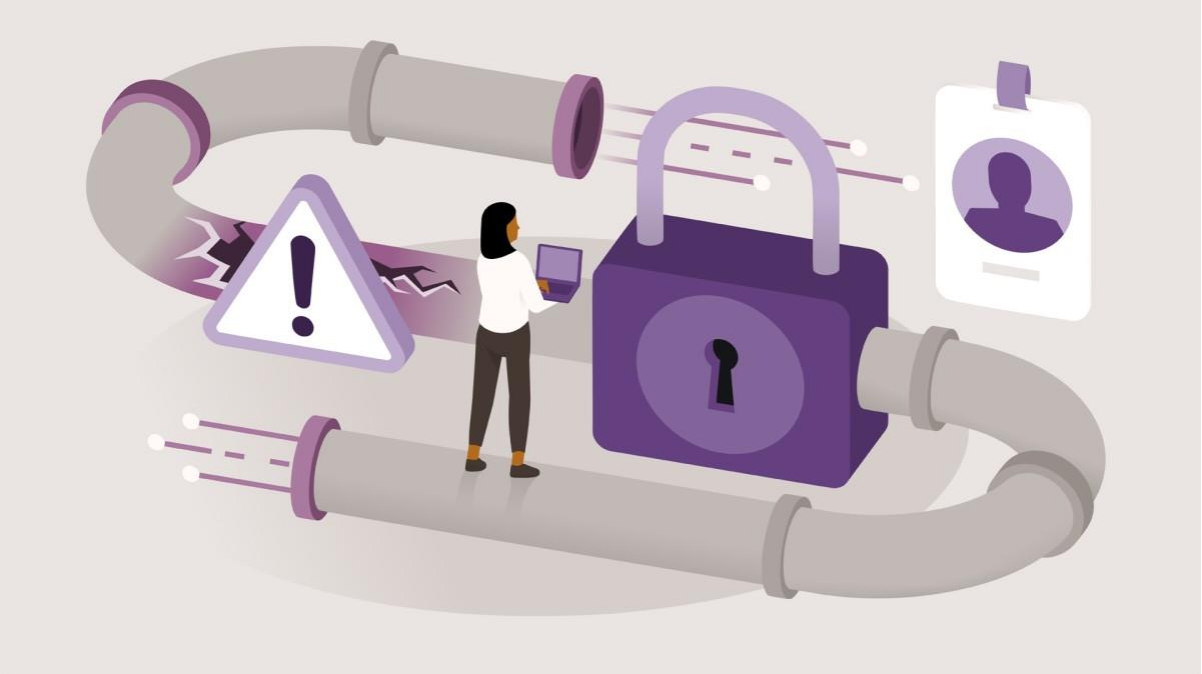
In simple terms, authentication failures happen when a system does not reliably verify a user's identity.

How could an attacker exploit it?

An attacker may:

- Try commonly used passwords.
- Use stolen credentials from previous data breaches.
- Perform credential-stuffing attacks using reused passwords.
- Brute-force weak passwords. • Hijack poorly protected sessions.
- Exploit weak password-reset mechanisms.

If successful, the attacker can access the victim's account as if they were the legitimate user



Real-world example:

The 2023 23andMe data breach involved credential-stuffing attacks. Attackers used usernames and passwords that had been exposed elsewhere and reused by users on the 23andMe platform.

The attackers gained access to accounts and used features connected to those accounts to obtain information related to a much larger number of users.

This incident showed that even when attackers do not directly break a company's password database, weak authentication protections and password reuse can still lead to serious data exposure.

How can developers prevent it?

Developers should

- Support and encourage multi-factor authentication (MFA).
- Require strong password policies without encouraging predictable patterns.
- Protect login endpoints against brute-force and credential-stuffing attacks.
- Rate-limit repeated login attempts.
- Securely store passwords using modern password-hashing algorithms.
- Use secure session management and short-lived sessions where appropriate.
- Detect unusual login behavior.
- Provide users with alerts for suspicious account activity

Personal Reflection:

The vulnerability that surprised me the most was Software Supply Chain Failures. I was surprised by how an organization can have strong internal security but still become compromised through a trusted software vendor or dependency. The SolarWinds attack especially showed how one compromised update can affect thousands of organizations. This made me realize that cybersecurity is not only about protecting your own code, but also about understanding and securing everything your systems depend on.

References:

1. OWASP Foundation https://owasp.org/Top10/2025/
2. https://youtu.be/Jzr0Jdnq_EI?si=e3OXkWhgraJtb40R
3. Meta Platforms, Inc. (2018-09-27) - The Cyber Security Incident Database (CSIDB)
4. https://prmia.org/common/Uploaded%20files/eAI/PRMIA%20Case%20study%20-%20Capital%20One.pdf

5. SolarWinds Data Breach: Supply Chain Attack Case Study
6. 2015 TalkTalk dat breach - Wikipedia
7. 23andMe Data Breach: How Credential Stuffing Exposed Genetic Data